

Use Critical Sections (Preferably Locks) to Eliminate Races

In a race, no one can hear you scream

By Herb Sutter

Herb is a software architect at Microsoft and chair of the ISO C++ Standards committee. He can be contacted at www.gotw.ca.

Everyone knows the basics of how to use locks:

```
mut.lock();    // acquire lock on x
... read/write x ...
mut.unlock();  // release lock on x
```

But why do locks, lock-free styles, and other synchronization techniques work at all, never mind interoperate well with each other and with aggressive optimizers that transform and reorder your program to make it run faster? Because every synchronization technique you've ever heard of must express, and every optimization that may ever be performed must respect and uphold, the common fundamental concept of a critical section.

Data Race

A data race (or just "race") occurs whenever the same memory location can be accessed simultaneously by more than one thread, and at least one of the accesses is a write. Consider the following code, where *x* is a shared object:

```
Thread 1      Thread 2
x = 1;  obj.f( x );
```

Thread 1 writes *x* and *Thread 2* reads it, and so this is a classic race if there is no synchronization that prevents the two threads from executing at the same time.

How potentially bad it can be to have a race? Very, depending on your language's and/or platform's memory model, which sets forth limits on compiler and processor reordering and on what, if any, guarantees you can expect. For example, in Java, "very strange, confusing, and counterintuitive behaviors are possible," including seeing partly constructed objects. [1] In POSIX threads (pthreads), there is no such thing as a benign race: Nearly any race could in principle cause random code execution.

Clearly, we want to eliminate races. The right way to do that is to use critical sections to prevent two pieces of code that read or write the same shared object from executing at the same time. But note:

- Not every object is shared for its entire lifetime: You only need to protect it with critical sections while it is shared, or while it is in transition from unshared to shared and vice versa (for instance, when a previously private object is first "published" to other threads by being made reachable outside its original thread).
- A shared object need not be protected in the same way for its entire lifetime: It is perfectly legitimate to protect the same object using locks at some times and lock-free techniques at others. Next month, I'll consider some examples in detail.

Different Ways to Spell "critical section"

A "critical section" is a region of code that shall execute in isolation with respect to some or all other code in the program, and every kind of synchronization you've ever heard of is a way to express critical sections. Today's most common synchronization tool is the humble lock, and every region of code that executes under a lock is a critical section. For example, consider again the earlier code:

```
mut.lock();    // enter critical section (a
... read/write x ...
mut.unlock(); // exit critical section (re
```

The next most common synchronization tools, used by wizard guru experts only, are varieties of lock-free coding. Beyond a handful of well-known patterns such as Double-Checked Locking, lock-free styles are typically too hard to use directly in normal programming, and they are usually used inside the implementations of other abstractions (for example, in the implementation of a mutex class, an internally synchronized lock-free hashed container, or an OS kernel facility).

The first lock-free style uses atomic variables (Java/.NET *volatile*, C++0x *atomic*) that enjoy special semantics with compiler and processor support. Consider an example similar to the aforementioned code, but written in a lock-free style, where *myTurn* is an atomic variable protecting *x*:

```
while( !myTurn ) { } // enter critical section (spin read)
... read/write x ...
myTurn = false;      // exit critical section (write)
```

The second lock-free coding style is to use explicit fences (also called "barriers") such as Linux *mb()*, or special functions with ordering semantics such as Win32 *InterlockedCompareExchange*. These tools express ordering constraints by placing explicit checkpoints where key variables are used in your code. Protecting a shared variable using these tools is difficult because the fences have to be written correctly at every point you use the variable (as opposed to once at the declaration of the variable to declare it inherently atomic), and their semantics are subtle and tend to vary from platform to platform. Here is one way to write the corresponding example, where *myTurn* is now just an ordinary variable (that must still be readable and writable atomically) and is given special semantics by applying a simple fence at each point of use so that it still correctly protects *x*:

```
while( !myTurn ) { } // enter critical section
mb();               // (atomic spin read + fence)
... read/write x ...
mb();               // exit critical section
myTurn = false;     // (fence + atomic write)
```

The key point is that all lock-based and lock-free styles are just different ways to express the same fundamental concept—the exclusively held critical section.

Memory Reordering

Figure 1 shows the canonical form and rules governing a critical section. Like any transaction, a critical section follows the basic acquire-work-release structure.

[Click image to view at full size]

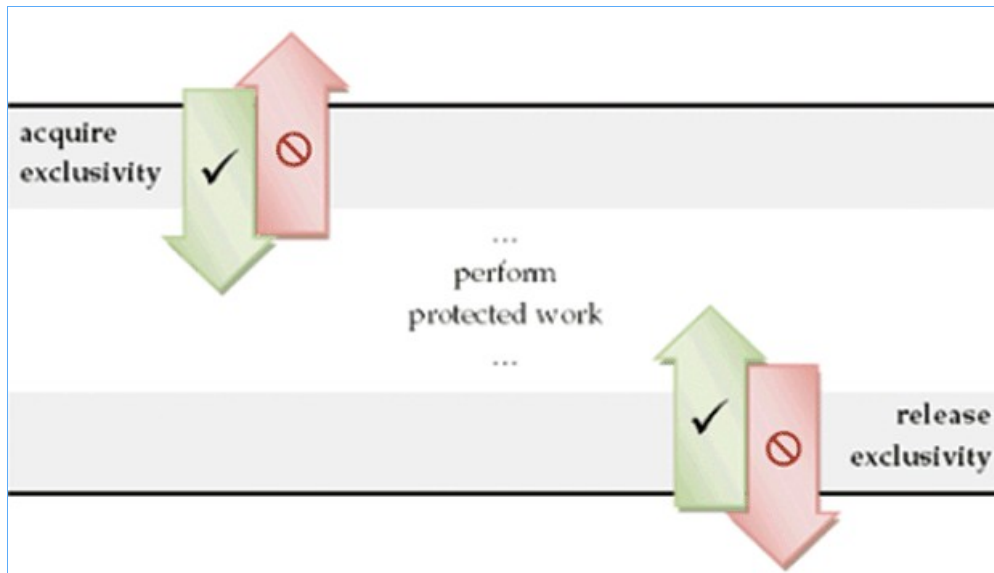


Figure 1: Anatomy of a critical section.

Compilers and processors routinely execute your code out of the order you specified in your source file, in order to make it run faster. For example, compiler optimizers want to help you by hoisting invariant calculations out of a loop; that means moving the instructions out of the loop body and actually executing them before the beginning of the loop. Processors want to help you by hiding the cost of accessing memory; that means moving expensive memory instructions ahead so that they can start sooner and overlap by having several in flight at the same time.

All of this reordering is fine as long as your program can't tell the difference—and the definition of "the program can't tell the difference" is "everybody respects the critical sections." Specifically:

- The programmer shall correctly eliminate races using critical sections.
- All reordering transformations shall respect the critical sections (and normal sequential control/flow dependencies as has always been done for plain old sequential optimizations).

So, for a reordering transformation to be valid, it must respect the program's critical sections by obeying the one key rule of critical sections: *Code can't move out of a critical section. (It's always okay for code to move in.)* We enforce this golden rule by requiring symmetric one-way fence semantics for the beginning and end of any critical section, illustrated by the arrows in Figure 1:

- Entering a critical section is an *acquire* operation, or an implicit acquire fence: Code can never cross the fence upward, that is, move from an original location after the fence to execute before the fence. Code that appears before the fence in source code order, however, can happily cross the fence downward to execute later.
- Exiting a critical section is a *release* operation, or an implicit release fence: This is just the inverse requirement that code can't cross the fence downward, only upward. It guarantees that any other thread that sees the final release *write* will also see all of the *writes* before it. [2]

Code Must Never Move Out

Let's see what "code can't move out" means in the context of the following code, which acquires a mutex *mut* that protects two integers *x* and *y*:

```
mut.lock();    // enter ("acquire")
```

```

        // critical section
x = 42;    // where can this line
           // appear to move to?
y = 43;
mut.unlock(); // exit ("release")
           //critical section

```

What are the legal reorderings of the assignment to *x*? It is perfectly legal for the system to transform the above code to:

```

mut.lock();
y = 43;
x = 42;    // ok: can move down
           // past y's assignment
mut.unlock();

```

because both assignments still happen inside the protected critical section (and do not otherwise depend on each other, assuming *x* and *y* are independent and not aliased). A system may not, however, transform the code to either

```

x = 42;    // invalid: race bait
mut.lock();
y = 43;
mut.unlock();

```

or

```

mut.lock();
y = 43;
mut.unlock();
x = 42;    // invalid: race bait

```

because either of these would move the assignment outside the critical section and therefore create a potential race on *x*.

So what about moving code *into* a critical section?

It's Okay For Code to Move In

Consider an adapted example:

```

x = "life";    // where can this line
               // appear to move to?
mut.lock();    // enter ("acquire")
               // critical section
y = "universe";
mut.unlock(); // exit ("release")

```

```
        // critical section
z = "everything"; // where can this
                // line appear to
                // move to?
```

What are the legal reorderings of the assignments to x and z ? Again assuming that x , y , and z are independent and not aliased, it is perfectly legal for the system to transform the above code to:

```
mut.lock();
z = "everything"; // ok: can move as
                // far up as this
y = "universe";
x = "life";      // ok: can move as
                // far down as this
mut.unlock();
```

Even though the moved lines now run while holding the lock, it doesn't alter the meaning or correctness of the code. It is always safe to add extra guarantees; in this case, to hold a lock a little longer.

But it is not safe to arbitrarily remove guarantees, such as to fail to hold a needed lock. Therefore, a system cannot arbitrarily reorder the code to cross either fence the wrong way:

```
z = "everything"; // invalid: race bait
mut.lock();
y = "universe";
mut.unlock();
x = "life";      // invalid: race bait
```

Note that this is true even though the assignments to x and z were not initially inside the critical section. For example, what if setting $y = \text{"universe"}$ is treated as a flag that tells another thread that x is now ready to be shared, so that y publishes x ? That is why no code must pass the release fence in the downward direction. [3]

Automate Acquire/Release

The whole system has to play by the rules—meaning the rules you wrote into your program. In particular, acquire and release fencing rules have to apply at every point beyond the source code, because when your program does strange things, it doesn't matter whether it was your compiler that reordered your statements or your processor that reordered the instructions emitted by the compiler. At the processor level, the only way to avoid instruction reordering is to use acquire- and release-style fences that most processors support as explicit standalone instructions. But you don't want to be in the business of writing fences by hand in your source code. So what can be done to control reordering?

Make the compiler both obey the rules and emit the right processor fences for you in one fell swoop: Use abstractions that express critical sections; namely, locks and atomic objects. Consider our first example of lock-based code, and how a compiler might translate it to specific "load-acquire" and "store-release" instructions when compiling for an IA64 processor:

```
mut.lock();
// "acquire" mut => ld.acq mut.var
... read/write x ...
mut.unlock();
// "release" mut => st.rel mut.var
```

But what about lock-free code? Similarly, for our other opening example of lock-free styles, we get:

```
while( !myTurn ) { }
// "acquire" => ld.acq myTurn
... read/write x ...
myTurn = false;
// "release" => st.rel myTurn
```

Both styles end up generating similar instructions because they express the same concept of a critical section. So avoid writing fences by hand; use these abstractions, and let the compiler write them for you.

Summary

Protect all mutable shared objects from races by putting code that accesses them into critical sections. The critical section is a fundamental concept that applies equally to all kinds of synchronization: Entering a critical section—taking a lock or reading from an atomic variable—is an acquire operation. Exiting a critical section—releasing a lock or writing to an atomic variable—is a release operation.

Next month, I look at practical examples of how, and how not, to use critical sections, including combining different styles.

Notes

[1] J. Manson, W. Pugh, and S. Adve. "JSR-133: Java Memory Model and Thread Specification" (*Java Community Process*, 2004).

[2] Note that I'm not specifying whether two consecutive critical sections could overlap, by allowing the "release" end of the first critical section to pass the "acquire" start of the second critical section. Some memory models allow this, whereas others have an additional requirement that acquire and release operations can't pass each other. This design choice doesn't affect the examples in this article.

[3] People regularly propose schemes that are more finegrained and try to let the programmer specify which objects are actually significant and must respect the fence. These have not become very popular, at least not yet, primarily because doing this adds great complexity to the programming model in return for insufficient actual performance benefit in most use cases.